

Build Manager Desktop Tool — Functional Requirements

Executive Summary

A native macOS desktop application (Tauri) that manages builds and workflows across multiple GitHub projects. Core function: user points to a repo, the tool scans it, discovers what can be built, displays available options, and executes builds with live output streaming.

Functional Requirements (What the Tool Does)

1. Repository Input & Detection

1.1 Open a Repository

User can input a GitHub repository URL OR a local folder path.

GitHub URLs: `https://github.com/user/repo`

Local paths: `/Users/user/projects/my-app` or `/Users/user/workspace/lxc-myhealthhub`

When a repo is entered:

App validates the input (URL or path format)

If GitHub URL: fetches repo contents from GitHub API

If local path: reads from filesystem

App scans the repository root for `/build` folder

If `/build` exists: auto-detection proceeds

If `/build` does not exist: displays error "No `/build` folder found in this repository"

1.2 Auto-Detect Build Structure

App automatically discovers what CAN be built in the repository.

When `/build` folder is found:

Scans `/build/scripts/` for executable files (`.sh`, shell scripts, or binaries)

Reads filenames as command names (e.g., `build-ios.sh` → "build-ios")

Optional: Reads script headers/comments to extract friendly names

Lists all discovered scripts as available build commands

Stores this information in app memory (tied to the repo)

If `/build/scripts/` is empty:

Displays message "No build scripts found in `/build/scripts/`"

User can still browse other folders in `/build/` if needed

1.3 Repository History & Persistence

App remembers accessed repositories across sessions.

Each time a repo is opened, it's added to "Recent Repositories" list

Recent list persists when app closes and reopens

User can re-open a recent repo without re-entering the URL/path

Optional: Show last accessed date for each repo

User can remove a repo from the history (clears from app, does not delete the actual folder)

1.4 Multiple Repositories

User can manage multiple repositories in the same session.

Can open/switch between different repos

Each repo maintains its own list of discovered build scripts
Switching repos updates the available build options
Build history is kept per-repo (optional: show which repo a build came from)

2. Build Execution & Management

2.1 Discover Available Build Commands

App automatically finds all executable build scripts.

When a repo is loaded:

Scans `/build/scripts/` directory

Lists all executable `.sh` files or binaries found

Extracts script name and converts to readable label:

`build-ios.sh` → "build-ios"

`release.sh` → "release"

`build-android.sh` → "build-android"

Optional: Reads script header comments (`# Build iOS app`) for friendlier display names

Displays each discovered script as a clickable button/option

2.2 Execute Build Scripts

User clicks a command button to run the associated build script.

When a build button is clicked:

App executes the script: `bash /path/to/repo/build/scripts/script-name.sh`

Working directory is set to the repository root

Captures all output (stdout and stderr)

Runs the script as a subprocess, not in an interactive terminal

2.3 Stream Live Output

User sees build output in real-time, line by line.

While a build runs:

Every line of output is captured and displayed immediately

Output appears in a designated log/output area

User can scroll through output if it exceeds visible area

Display is non-blocking (user can scroll while build continues)

Each output line is timestamped (e.g., `[14:32:45] Compiling source...`)

2.4 Build Status Tracking

App tracks whether a build is running, succeeded, or failed.

For each build attempt:

In Progress: Visual indicator (spinner, "Building..." text) shows build is running

Completed (Success): Indicator changes to checkmark + duration (e.g., "✓ Completed in 4m 32s")

Completed (Failed): Indicator shows error state + duration (e.g., "× Failed after 2m 15s")

Duration: App measures time from start to finish and displays it

Status persists on screen until user starts a new build or clears it

2.5 Cancel Running Build

User can stop a build that's currently running.

While a build is in progress, a "Cancel" or "Stop" button is visible

Clicking it terminates the script process immediately

Any partial output generated before cancellation is preserved

Status indicator updates to "Cancelled"

3. Log Storage & Retrieval

3.1 Log File Persistence

App saves build logs to disk for later review and archival.

Each time a build runs:

A new log file is created in repository's `/build/logs/` directory

Filename format: `build-YYYY-MM-DD-HH-MM-SS.log` (e.g., `build-2026-08-16-14-32-45.log`)

Filename includes: script name, timestamp, status (success/failure)

Full output (stdout + stderr) is written to the log file

Log file is human-readable plain text

3.2 View Build Logs

User can view current and past build logs in the app.

Current Build: Display live output of the running/most-recent build

Past Builds: User can select a previous build from history to view its complete log

Log Display Format:

Monospace font (Monaco, Courier, etc.)

Terminal-style (dark background, light text)

Each line shows timestamp + output text

Display is scrollable

3.3 Search Within Logs

User can search for specific text or errors in a log.

Text input field to search within the current log

Matching lines are highlighted

Search results show count (e.g., "3 matches")

User can navigate between matches (next/previous buttons)

3.4 Filter Logs

User can filter log output by type.

Optional features:

"Errors Only" — show only lines containing error keywords (error, Error, ERROR, failed, Failed)

"Warnings Only" — show only warnings (warning, Warning, WARNING)

"Info Only" — show only informational lines

Filters apply to currently displayed log

3.5 Export Logs

User can export a build log to their computer.

"Export" or "Download" button for current log

Saves log file to user's Downloads folder (or user-selected location)

File format: plain text (`.log`) or optionally as `.txt`

4. Project & Build Overview

4.1 Show Current Repository Info

Display key information about the currently loaded repository.

When a repo is loaded, show:

Repo Name: Clear display of which project is active

Repo Path/URL: Full path (for local) or GitHub URL

Connection Status: Visual indicator (✓ connected, × unreachable)

Total Builds Run: Count of builds executed in this app session for this repo

4.2 List All Available Build Commands

Show all discovered build scripts as executable options.

Display all scripts found in `/build/scripts/` with:

Script Name: Human-readable label

Last Run Info: When this script was last executed (if ever)

e.g., "Last run: 2 hours ago ✓"

e.g., "Last run: 5 days ago ×" (with warning if >3 days old)

e.g., "Never run" if no history

Status Badge: Visual indicator of last run (success/failure/never)

4.3 Track Build History Per Repository

Keep a record of all builds run on each repository.

For each repo:

Store list of all builds run (script name, timestamp, status, duration)

Show count of total builds run

Show success rate (e.g., "18 successes, 2 failures = 90% success rate")

Show average build duration (e.g., "Average: 5m 12s")

4.4 Quick Statistics

Show summary metrics for the current repository.

Display cards or stats showing:

Total Builds This Session: "12 builds run"

Success Rate: "92% (11 success, 1 failed)"

Most Recently Run: "Build iOS — 15 minutes ago"

Last Failed Build: "Build Android — 2 days ago" (if any failures exist)

5. Multi-Repository Support

5.1 Switch Between Repositories

User can work with multiple repos in the same session.

App displays a list of all opened/recent repositories

User can click on a repo to switch to it

When switching:

Build buttons update to show that repo's scripts

Log display clears (ready for new builds)

Status info updates to that repo's stats

History switches to that repo's build history

Switching is instant (no loading screens)

5.2 Track Build History Per Repository

Each repository maintains its own build history.

When user runs a build, it's recorded under that repo

Switching repos shows that repo's build history (not mixed with other repos)

Each build record includes: timestamp, script name, status, duration, full log

5.3 Manage Multiple Repositories

User can add, remove, and organize multiple repos.

Recently accessed repos appear in a list for quick re-access

User can "favorite" or "pin" frequently-used repos

User can remove a repo from the app (clears from list, doesn't delete the actual folder)

Each repo in the list shows its name and URL/path

Technical Stack

Framework: Tauri (Rust backend + web frontend)

Frontend: React or vanilla JS + CSS

Backend: Rust (file system access, GitHub API, process execution)

APIs: GitHub REST API v3 (for repo content fetching)

Storage: Local JSON config for saved repos & history

Logging: File-based logs in `/build/logs/` directory

User Workflows

Workflow 1: Open a Repository & Run a Build

Launch app → App opens showing welcome/home screen

User pastes repo URL or path → `https://github.com/user/lxc-myhealthhub` or `/Users/user/workspace/lxc-myhealthhub`

App validates input → Checks if URL/path is valid

App scans `/build` folder → Finds all `.sh` files in `/build/scripts/`

App displays build options → Shows clickable buttons for each discovered script (Build iOS, Build Android, Release, etc.)

User clicks "Build iOS" → Build starts executing

Live output streams → Each line of output appears in real-time with timestamps

Build completes → Status changes to "✓ Success" + duration, log is saved

User clicks "Release" → Next build starts, repeating the cycle

Workflow 2: Manage Multiple Repositories

User has Repo A open

User clicks "Add Repository" → Opens input dialog

User enters Repo B URL → App loads Repo B

User sees Repo B's build options → Scans `/build` and shows available commands

User switches back to Repo A → List of repos shows both A and B

User clicks Repo A in list → Switches back to Repo A, shows Repo A's build history and options

Workflow 3: View Build Logs

User runs a build → Output streams and log is saved to `/build/logs/build-2026-08-16-14-32-45.log`

Build completes → Full log displayed in app

User wants to search the log → Types "error" in search box

App highlights matching lines → Shows "3 matches found"

User clicks "Export" → Log is saved to Downloads folder as plain text file

Workflow 4: Track Build History

User has run 5 builds in this repo → All 5 are recorded with timestamps and status

User views "History" tab/section → Shows chronological list:

14:45 Build iOS ✓ (4m 32s)

14:30 Build Android × (2m 15s)

14:00 Release ✓ (8m 01s)

13:15 Build Docs ✓ (1m 45s)

12:00 Build iOS ✓ (4m 28s)

User clicks Build Android (failed) → Previous log is displayed, showing why it failed

Non-Functional Requirements

Performance: App launches in <2 seconds, repo scan in <5 seconds

Reliability: Graceful error handling for network issues, missing repos

Scalability: Support 5-10+ connected repos without performance degradation

Maintainability: Simple codebase, minimal dependencies, easy to extend

Offline Support: Can view logs and history offline; GitHub fetch fails gracefully

Scope (v1)

- ✓ Repo URL input + history
- ✓ `/build` folder detection
- ✓ Button generation from detected scripts
- ✓ Build execution + log streaming
- ✓ Multi-project support
- ✓ Native macOS `.app` package
- ✓ Basic dashboard UI

Out of Scope (v2+)

GitHub Actions integration

Webhook notifications

Slack/email alerts

Docker build support

Windows/Linux versions (v2)

Advanced analytics/charts

Deliverables

Working Tauri project (macOS)

Source code repository

Build instructions (how to package `.app`)

User guide / quick start

Configuration template (`projects.json`)